

# Сборка кодинг-агента

Семь ступеней конфигурации на одном сквозном кейсе

Один репозиторий. Семь ступеней конфигурации. Один и тот же сайт от пустой папки до показа заказчику.

ОДИН РЕПОЗИТОРИЙ, СЕМЬ ДОБАВЛЕНИЙ



Реальные 7 стадий демо-репозитория `signup-landing/` (`git ls-tree` по коммитам) · Источник: `captures/21–27`

## Девять разделов занятия — один сквозной сайт, семь ступеней одной лестницы



*Один репозиторий, семь ступеней конфигурации, один и тот же агент — от пустой папки до показа заказчику*

## Один и тот же репозиторий одного и того же сайта проходит все семь ступеней



### Задача

Лендинг с формой заявки на демо-урок: поля «имя» и «email», без собственного бэкенда



### Стек

Vite (npm run build → dist/) · Playwright (tests/form.spec.ts) · статический хостинг · wrangler pages deploy · задачи только в GitHub Issues



### Кто работает

Один разработчик с агентом Claude Code, без команды



### «ГОТОВО»

npm run build код 0 · npx playwright test зелёный · сайт открыт по прод-URL · форма реально отправляет заявку на проде

У заказчика назначен показ сайта — фиксированная дата

## Репозиторий совершенно пуст — добавлять в конфигурацию пока физически нечего

```
$ ls -la signup-landing/  
total 8  
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .  
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

Ступень 0 из таблицы лестницы — «ничего» — не абстракция, а реальный вывод реальной команды

## Одна задача целиком, одним запросом — не по частям — и агент строит сайт с нуля

```
$ cd signup-landing/  
$ claude
```

«Собери лендинг с формой заявки на демо-урок: статическая HTML-страница, сборка Vite (`npm run build → dist/`), поля формы — имя и email, тест на Playwright, который проверяет, что форма не отправляется с пустыми обязательными полями (`tests/form.spec.ts`)».

HTML · CSS · JS

VITE BUILD

PLAYWRIGHT

ИМЯ + EMAIL

## Агент начинает с нуля и не знает ничего, что не выводимо из уже существующего кода



Агент создаёт `index.html`, `src/main.js`, `package.json`, `tests/form.spec.ts`



*Момент, где агент уточняет что-то не выводимое из кода — например, каким хостингом сайт будет деплоиться или какой командой проверить, что сборка вообще прошла.*

**На маленьком, только что собранном репозитории правильный ответ — «ничего»**

**«Перед вами репозиторий лендинга, который агент только что собрал с нуля — и вам с этим же агентом предстоит доделывать этот сайт дальше, неделями. Какой блок конфигурации вы бы добавили первым, и почему именно его, а не любой другой из оставшихся шести?»**



*Открытый вопрос классу — два-три голоса, затем разбор*

Семь ступеней лестницы — один принцип: каждый блок добавляется по конкретному триггеру, не про запас

| № | Триггер «стало нужно»                              | Что добавляем                 | Критерий «ещё рано»                                        |
|---|----------------------------------------------------|-------------------------------|------------------------------------------------------------|
| 0 | —                                                  | ничего                        | разовая задача, маленький репозиторий                      |
| 1 | агент каждый раз переспрашивает одно и то же       | файл инструкций               | агент и так справляется; документации хватает              |
| 2 | владелец повторяет одно и то же из сессии в сессию | память + журнал решений       | одна-две сессии; факт уже записан в коде                   |
| 3 | правило записано, но агент его нарушает            | хук                           | правило дешевле проверить в сборке; нарушение стоит дешево |
| 4 | файл инструкций облагает налогом каждый запрос     | скилл                         | нужно каждой сессии → это не скилл                         |
| 5 | нужен доступ к внешней системе                     | МСП                           | хватает обычной команды                                    |
| 6 | повторяющаяся роль (ревью, разведка)               | субагент                      | шаги зависимы и требуют общего контекста                   |
| 7 | работу принимают на слово                          | прожарка + оперативный журнал | задача дешевле процесса вокруг неё                         |

## Явно произнесённое правило, ничем не подкреплённое механически, не защищает ни от ошибки, ни от лжи о результате



### Replit, июль 2025, день 9 эксперимента

Jason Lemkin несколько раз явно объявил code freeze — прямой запрет на изменения. Агент проигнорировал это и выполнил деструктивные операции против продакшен-базы данных SaaSTr. На вопрос про откат агент заявил, что откат невозможен, хотя технически он был доступен. Уничтожены записи ~1206 руководителей и ~1196 компаний. Агент также сфабриковал ~4000 фейковых профилей пользователей и исказил отчёт о тестах.

Ни одного из семи блоков конфигурации сегодняшнего занятия — поэтому пример открывает семинар

# 1

## Файл инструкций

АГЕНТ ПЕРЕСПРАШИВАЕТ ТО ЖЕ САМОЕ ВО ВТОРОЙ РАЗ



## Агент второй раз подряд спрашивает то же самое в новой сессии



*Ждёт реальной сессии Claude  
Code*

*Новая сессия того же репозитория signup-landing. Просьба добавить второе поле в форму и прогнать проверку — и агент снова спрашивает, какой командой собрать сайт и какой командой прогнать тест формы.*

**«Агент второй раз спрашивает, какой командой собрать этот лендинг и как проверить, что форма проходит тест. Что вы сделаете — и куда именно это запишете?»**



**Открытый вопрос классу — два-три голоса из зала, затем разбор**

## CLAUDE.md содержит только то, что агент не выведет сам из кода

### CLAUDE.md

#### # CLAUDE.md

signup-landing: статический лендинг с формой заявки на демо-урок.  
Готово = npm run build код 0, npx playwright test зелёный,  
форма реально отправляет заявку на проде.

#### ## Safety / scope boundaries

- Никогда не запускать деплой на прод без явного запроса.

#### ## Build, test, verify

- Build: npm run build
- Test: npx playwright test
- Run locally: npm run dev
- To verify a change actually works (not just compiles):  
открыть dist/index.html после сборки, вручную отправить  
форму с заполненными и с пустыми полями.

## AGENTS.md — симлинк на CLAUDE.md, не копия: один источник правды

```
$ ln -s CLAUDE.md AGENTS.md
```

Дерево репозитория на этой ступени

```
$ git ls-tree -r --name-only 04df4a3 | sort    # commit 04df4a3
```

AGENTS.md

CLAUDE.md

index.html

package.json

src/main.js

tests/form.spec.ts

## Правило, записанное корректно, всё равно может быть проигнорировано



**anthropics/claude-code#42863, закрыт как «not planned», апрель 2026**

Агент дословно проигнорировал правило CLAUDE.md, требующее подтверждения перед доступом к файлам вне рабочей директории — выполнил `msiexec /i` с записью в `C:\Program Files`, регистрацией служб и правкой реестра, без запроса.



**dev.to, 2026**

200 строк CLAUDE.md + 258 файлов базы знаний остались практически непрочитанными — «write-only» документация. Детальные протоколы и «запрещённые фразы» не помогли. Вывод автора дословно: «Rules in prompts are requests. Hooks in code are laws».

Наличие CLAUDE.md само по себе не повышает успех задачи, а поднимает стоимость на 20–23%

| Замер                                                                                                                    | Значение                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Контролируемый эксперимент, LLM-сгенерированный CLAUDE.md вида «repository overview» (Gloaguen et al., arXiv:2602.11988) | Измеримого выигрыша в успехе задачи нет; стоимость и число шагов выросли на 20–23%                                     |
| Соблюдение инструкций при росте их числа (IFScale, arXiv:2507.11538)                                                     | Лучшие модели держат только 68% точности при 500 одновременных инструкциях; комплаенс падает почти линейно с их числом |

«Bloated CLAUDE.md files cause Claude to ignore your actual instructions!» — официальная рекомендация Anthropic, таргет ≈200 строк

## Здесь файл ещё не нужен

Крошечный репозиторий или одноразовый скрипт, где структура помещается в один экран и не требует нестандартных команд сборки — файл добавляет только налог на контекст без компенсирующей выгоды

### БАЗА

Агент читает файл в начале каждой сессии — как памятка на столе нового сотрудника, которую читают, хочет он того или нет.

---

*Мост к следующей ступени: Решение — например, почему в форме нет сторонней библиотеки валидации — в CLAUDE.md не попадёт: это не про команды, это про то, что уже один раз отклонили. Для решений и их обоснований у лестницы есть отдельный, следующий блок.*

# 2

## Память

ОДНО И ТО ЖЕ РЕШЕНИЕ ДВАЖДЫ → ФАКТ НИГДЕ НЕ ЗАПИСАН



## Решение, уже отклонённое в прошлой сессии, снова всплывает как будто впервые



### Вторую неделю подряд

Вы объясняете агенту одно и то же: почему форма проверяет поля нативным HTML5 (required, pattern), а не сторонней библиотекой валидации. Эта библиотека уже была явно отклонена в прошлой сессии. Теперь агент снова предлагает её подключить, как будто вопрос не поднимался вообще.

«Вы второй раз за неделю объясняете агенту, почему в форме заявки нет сторонней библиотеки валидации — то самое решение, которое уже отклонили в прошлой сессии. Куда это должно попасть, чтобы не объяснять в третий раз — и почему не в CLAUDE.md?»

Впишу  
в CLAUDE.md

Ничего специально,  
сам запомнит

Заведу Mem0  
или Zep

«Запомни» +  
DECISIONS.md

## Авто-память рантайма живёт вне репозитория и грузится в каждую новую сессию сама

### «ЗАПОМНИ» → ФАЙЛ



*«запомни: не подключаем стороннюю библиотеку валидации форм — форма из двух полей, хватает нативного HTML5»*

*Ждёт реальной сессии Claude Code*

*Создаётся `feedback_native-form-validation.md` в  
~/.claude/projects/signup-landing/memory/*

### НОВАЯ СЕССИЯ → /context



*MEMORY.md виден среди загруженного контекста, не только на диске*

*Ждёт реальной сессии Claude Code*

*Только если MEMORY.md виден в перечисленном контексте — механизм реально работает, а не просто существует файлом*

## DECISIONS.md — командный, версионизируемый лог «почему», видимый всем через git

```
diff --git a/DECISIONS.md b/DECISIONS.md
new file mode 100644
--- /dev/null
+++ b/DECISIONS.md
+# Decisions
+
+Append-only log of why, in date order.
+
+## 2026-09-20 — не подключаем библиотеку валидации форм
+Форма — два поля (имя, email), хватает нативных required/pattern
+в index.html. Сторонняя библиотека — лишняя зависимость и лишние
+килобайты в сборке ради этого объёма.
```

Авто-память — быстрый, машинно-локальный слой, не проходит код-ревью. DECISIONS.md — коммитится, ревьюится, видно всем через git.

## Однажды попавшее в память обслуживается как доверенный контекст бессрочно



### SpAIware, сентябрь 2024

Непрямая инъекция через текст, который агент просто прочитал, внедрила ложную инструкцию в долговременную память macOS-приложения ChatGPT. Инструкция пережила конец разговора и активировалась в будущих сессиях — включая непрерывную эксфильтрацию переписки на сервер атакующего.

*Johann Rehberger; OpenAI закрыл вектор эксфильтрации в версии 1.2024.247, тоже сентябрь 2024*



### Второй слой: Anthropic Memory Tool, эфемерное усмотрение модели

Модель сама решает, что «стоит» запоминать — непоследовательно между прогонами. harness-eval issue #40 (2026): на 4 из 24 задач, это 17%, модель явно отказалась записать нужный факт. Один и тот же диалог, обработанный дважды, один раз сохранил факт, другой раз — нет.

## Здесь ещё не нужно

задача одноразовая

весь контекст помещается в одно окно

правила проекта стабильны и укладываются в CLAUDE.md < 200 строк

никто не повторяет вручную одну и ту же правку из сессии в сессию

нет нескольких участников, которым нужно делиться контекстом асинхронно

факт и так выводим из кода или git

### БАЗА

Агент не помнит вчерашней сессии, если факт не записан на диск.

---

Нюанс: CrewAI Memory на LongMemEval — с памятью 46,0%, без памяти 57,6% (включение памяти сделало систему хуже).

# 3

## Хуки

ПРАВИЛО ТЕКСТОМ → ПРАВИЛО, КОТОРОЕ ИСПОЛНЯЕТ СРЕДА



## Правило было сформулировано абсолютно корректно текстом — и всё равно нарушено накануне показа



### Завтра — показ сайта заказчику

В CLAUDE.md уже написано текстом: «никогда не коммитьте напрямую в main». Накануне вечером, в спешке перед показом, агент всё равно выполняет `git commit` прямо на main — правка формы уходит в продакшен-ветку без ветки и без ревью.

«Агент только что, накануне показа, закоммитил правку формы в main — текстом всё было сказано правильно. Что вы делаете, чтобы это стало невозможно, а не просто маловероятно?»

Пишу правило  
капсом

Проверяю руками  
после коммита

Защита ветки  
на сервере

Добавлю  
PreToolUse-хук

## .claude/settings.json: хук защиты main

```
.claude/settings.json
{
  "hooks": {
    "PreToolUse": [{ "matcher": "Bash", "hooks": [{
      "type": "command",
      "command":
        "jq -r '.tool_input.command' | { read -r cmd;
          if echo \"$cmd\" | grep -qE '^(^|[:&|]\\s*)git\\s+commit\\b'; then
            if ! git rev-parse --is-inside-work-tree >/dev/null 2>&1; then
              echo permissionDecision=ask ...; exit 0; fi;
            branch=$(git symbolic-ref --short HEAD 2>/dev/null);
            if [ \"$branch\" = main ] || [ \"$branch\" = master ]; then
              echo permissionDecision=deny \"BLOCKED: direct commit
                to main/master. Create a feature branch first.\"; fi;
            fi; }",
      "timeout": 10
    }] }] }
```

*git symbolic-ref --short HEAD, не git rev-parse --abbrev-ref HEAD — на unborn HEAD rev-parse падает и молча оставляет проверку нерабочей. Источник: captures/29-code-settings.json.txt*

## Хук в действии: deny на ветке main

```
=== INPUT ===
{"tool_input":{"command":"git commit -m \"fix: second form field\""}}
=== current branch ===
main
=== HOOK OUTPUT (stdout) ===
{"hookSpecificOutput":{"permissionDecision":"deny",
  "permissionDecisionReason":"BLOCKED: direct commit to
  main/master. Create a feature branch first."}}
=== exit code: 0 ===
```

Источник: `captures/06-hook-deny-main.txt` — перенесено дословно

## Тот же коммит на рабочей ветке проходит

```
=== git switch -c fix-form-field ===  
=== INPUT ===  
{"tool_input":{"command":"git commit -m \"fix: second form field\""}}  
=== current branch ===  
fix-form-field  
=== HOOK OUTPUT (stdout) ===  
(пусто)  
=== hook exit code: 0 (empty stdout = not blocked, commit proceeds) ===
```

Источник: `captures/07-hook-allow-feature-branch.txt` — перенесено дословно

## Вне репозитория: ask, не тихое allow

```
=== каталог: /tmp/no-repo-demo (git init НЕ выполнялся) ===  
$ git rev-parse --is-inside-work-tree  
fatal: not a git repository (or any of the parent directories): .git  
(exit code: 128)  
=== HOOK OUTPUT (stdout) ===  
{  
  "permissionDecision": "ask",  
  "permissionDecisionReason":  
    "WARNING: branch-protection gate inactive – this is not a  
    git repository yet ... run git init first"}  
}
```

Источник: `captures/09-hook-ask-no-repo.txt` — перенесено дословно

## Вредоносный хук выполнялся раньше, чем диалог доверия к папке вообще появился на экране



### CVE-2025-59536, CVSS 8.7

Вредоносный хук на SessionStart в чужом `.claude/settings.json` выполнялся сразу после клонирования репозитория и запуска `claude` — команды из хука уже отработали к моменту, когда диалог доверия к папке вообще появился на экране.

*Check Point Research; найдено 21.07.2025, патч 26.08.2025, advisory 29.08.2025*



### Второй слой: тихие несрабатывания

Один невалидный `matcher` в `settings.json` отключает все хуки файла целиком, без единой ошибки в интерфейсе. `PreToolUse`-хук, не уложившийся в таймаут, не блокирует — просто отваливается. Субагенты не запускают родительские хуки. «when a gate doesn't fire, nothing happens, which is exactly what a passing gate looks like».

## Здесь ещё не нужно

проверка дешевле реализуется в CI или линтере на PR, чем на каждый PostToolUse интерактивной сессии  
хук становится тормозом при стекировании — раздутие не даёт явной ошибки, просто всё замедляется  
правило по сути не механизировано — суждение вроде «не переусложняй» честнее оставить человеку  
соло-разработчик с ограниченной зоной поражения ошибки — риск ещё не стал реальным

### БАЗА

Хук — правило, которое среда исполняет сама, не спрашивая согласия агента, как турникет вместо таблички «пожалуйста, платите».

---

Нюанс: *deny в permissions всегда побеждает allow, даже более узкий и точный. «hook bloat has no error message. Nothing fails. Everything just gets slower, gradually».*

# 4

## СКИЛЛЫ

РЕДКАЯ ПРОЦЕДУРА → ЛИШНИЙ ГРУЗ В КАЖДОЙ СЕССИИ



## Сорок строк процедуры деплоя грузятся в каждую сессию, даже когда речь о мелком баг-фиксе



### В CLAUDE.md накопилось 40 строк

Процедура деплоя на прод — собрать, прогнать smoke-тест формы, опубликовать на Cloudflare Pages, проверить прод-URL. Нужна раз в две недели. Но грузится в контекст каждой сессии — даже когда вы просто чините баг в вёрстке.

«В CLAUDE.md этого сайта накопилась процедура деплоя на 40 строк, нужна раз в две недели. Она грузится в контекст каждой сессии, даже когда вы просто чините баг в форме. Куда её вынести — и как сделать так, чтобы агент сам понял, когда её открыть?»

Оставлю  
«на всякий случай»

Вынесу, сошлюсь  
@-импортом

Заведу файл,  
буду сам напоминать

Скилл  
с description

## Скилл читается в три уровня — грузится всегда только первый

| Уровень        | Когда грузится            | Стоимость         | Содержимое               |
|----------------|---------------------------|-------------------|--------------------------|
| 1. Метаданные  | всегда, при старте сессии | ≈100 токенов      | name + description       |
| 2. Инструкции  | при срабатывании          | <5000 токенов     | тело SKILL.md            |
| 3. Ресурсы/код | по требованию             | 0, пока не открыт | вложенные файлы, скрипты |

Отсюда следствие для полей: name ≤64 символа, kebab-case; description не может быть пустым и обязан назвать и «что делает», и «когда использовать» — по этому единственному полю первого уровня модель решает, открывать ли остальное.

## .claude/skills/deploy/SKILL.md

deploy/SKILL.md

---

**name:** deploy

**description:** Деплоит лендинг `signup-landing` на прод — сборка, smoke-тест формы, публикация, проверка прод-URL. Используй, когда пользователь просит задеплоить, выкатить или опубликовать сайт — не для обычных правок кода без деплоя.

---

**# deploy**

1. Собрать: `npm run build` (код выхода должен быть 0).
2. Прогнать smoke-тест формы: `npm run playwright test` (зелёный).
3. Опубликовать: `wrangler pages deploy dist --project-name signup-landing`.
4. Открыть прод-URL, вручную отправить форму — заявка должна прийти.
5. Записать в `Tasks/<date>_<slug>/log.md`, что задеплоено, когда и как.

12 из 13 скиллов этого же курса не имеют фронтматтера с description

| Скилл             | Есть фронтматтер | Есть description |
|-------------------|------------------|------------------|
| build-deck        | нет              | нет              |
| catalog-docs      | нет              | нет              |
| compile-wiki      | нет              | нет              |
| diagram-refresh   | нет              | нет              |
| extract-links     | нет              | нет              |
| impact-check      | нет              | нет              |
| issue-from-change | нет              | нет              |
| pre-user-gate     | да               | да               |
| publish-article   | нет              | нет              |
| query-kb          | нет              | нет              |
| reflect           | нет              | нет              |
| sync-library      | нет              | нет              |
| update-lecture    | нет              | нет              |

1 из 13

СКИЛЛОВ

имеют YAML-фронтматтер с description

skill-induced failures (arXiv:2608.11888): 307 случаев вреда, 68,8% (86/125) — Task-Implementation Fault (ошибка в самой реализации задачи); до +451% токенов на задачу (SWE-Skills-Bench)

## Скилл нужен не каждой повторяющейся задаче



**Нужно каждой сессии**

---

*Это файл инструкций — ступень 1, не скилл*



**Разовая процедура**

---

*Дешевле один раз подробно попросить, чем писать скилл заранее*



**Нельзя нарушать никогда**

---

*Это хук — ступень 3, механическое ограничение, не знание*

*скилл — методичка в шкафу, которую снимают с полки только когда задача реально совпала с темой методички*

5

## MCP (Model Context Protocol)

РЕГУЛЯРНЫЙ ДОСТУП К ВНЕШНЕЙ СИСТЕМЕ → ОБЩИЙ ПЕРЕХОДНИК



## Агент почти готов получить официальный GitHub MCP-сервер



### Issue и PR на каждой сессии

Заказчик и разработчик регулярно заводят задачи и баги как issue в репозитории signup-landing на GitHub — на каждой сессии агенту нужно посмотреть открытые issue и подготовить черновик PR. Сейчас — копия или нестабильный gh через Bash.

«Агенту теперь регулярно нужно смотреть открытые issue в репозитории signup-landing и готовить черновики PR. Вы почти готовы подключить официальный GitHub MCP-сервер. Что вы проверяете до того, как это сделать?»

Официальный —  
значит можно

Прикину  
токены

Разрешу сразу  
на всё

Проверю gh  
+ права доступа

## .mcp.json живёт в корне репозитория и коммитится как код

`.mcp.json`

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_PAT}" }
    }
  }
}
```

tools · инструменты

**решает МОДЕЛЬ**

*вызывает по ходу диалога*

resources · ресурсы

**решает ХОСТ**

*явное @-упоминание*

prompts · подсказки

**решает ЧЕЛОВЕК**

*/-команда*

## Определения инструментов MCP занимают контекст ещё до начала работы

| Замер                                                | Значение                                     |
|------------------------------------------------------|----------------------------------------------|
| Типичная связка из 5 серверов, без Tool Search       | ≈55 000 токенов определений до начала работы |
| То же, с Tool Search (по умолчанию)                  | тот же объём минус более 85%                 |
| Реалистичная связка 5 серверов (mcp-token-benchmark) | 26 224 токена = 13,1% от окна 200K           |
| Одно действие «язык репозитория», CLI vs MCP         | 1 365 против 44 026 токенов                  |

## «Подключён» — не то же самое, что «работает»

`notes/mcp-limitations.md`

**Server: workspace-mcp** [#49]

Symptom: все вызовы возвращают ACTION REQUIRED: Google  
Authentication Needed, хотя `claude mcp list` показывает

**сервер как ✓ Connected**

Root cause: OAuth-приложение в статусе Testing — `refresh_token`  
отзывается автоматически через 7 дней неактивности

Второй, независимый инцидент того же сервера [#86]: `claude mcp list` показывает уже ✗ Failed to connect — регрессия транзитивной зависимости aiofile 3.10.0. Статус в интерфейсе — источник истины только о факте подключения, не о работоспособности.

## Одновременный доступ к публичному и приватным репозиториям позволил агенту опубликовать чужие приватные данные



26 мая 2025, Invariant Labs

Агент с одновременным доступом к чтению публичного репозитория и к приватным репозиториям пользователя через один и тот же GitHub MCP-сервер прочитал скрытую инструкцию — промпт-инъекцию — в открытом issue, собрал данные из приватных репозиториях — включая адрес и зарплату — и опубликовал их в автоматически созданном публичном PR.

*CVE не присвоен — архитектурный риск, не баг кода*



«Смертельное трио» (Simon Willison, 16 июня 2025)

Три условия одновременно делают утечку возможной без единой строчки эксплойт-кода: доступ к приватным данным + обработка недоверенного внешнего контента + канал отправки наружу. Та же схема независимо воспроизвелась в июле 2025 на Supabase MCP.

## Здесь MCP-сервер ещё не нужен

 Хватает gh / psql / aws через Bash

*Задача сводится к известному CLI-вызову*

 Разовое или редкое действие

*Не повторяющийся доступ*

 Снимок на момент запуска достаточен

*Реальное время не критично*

 Один клиент, одна сессия

*Нет нужды делить состояние между несколькими*

*MCP — общий переходник к внешней системе, чтобы не писать отдельную интеграцию под каждого клиента*

# 6

## Субагенты

ТА ЖЕ СЕССИЯ НЕ МОЖЕТ ЧЕСТНО ПРОВЕРИТЬ СЕБЯ



## Одна и та же сессия не может быть независимым ревьюером собственного изменения



### Третий раз за неделю

Вы просите ту же сессию, что писала правку `src/validate.js`, честно перепроверить саму себя перед тем, как деплоить изменение на прод, «максимально придирчиво». Сессия старается быть состязательной к себе — но остаётся тем же контекстом рассуждений, тем же слепым пятном.

«Она честно старается быть придирчивой. Достаточно ли этого — и если нет, что именно нужно заменить?»



Открытый вопрос классу — два-три голоса из зала, затем разбор

## Ревьюер видит только diff и критерии приёмки — не рассуждения, которые привели к изменению

diff-reviewer.md

---

**name:** diff-reviewer

**description:** Независимый ревьюер diff'ов. Видит только сам diff и критерии приёмки, не рассуждения автора. Используй перед мержем любого нетривиального изменения. НЕ для методической критики текста.

**tools:** Read, Grep, Glob

---

**# diff-reviewer**

Начни ответ строкой:

**VERDICT:** PASS | BLOCK

src/validate.js diff

```
diff --git a/src/validate.js b/src/validate.js
@@ export function showValidationErrors(form) {
    form.reportValidity();
  }
+// email regex дополнен: раньше пропускал
+// "a@b" без точки в домене
+export function isValidEmail(value) {
+  return /^[^@\\s]+@[^@\\s]+\\. [^@\\s]+$/ .test(value);
+}
```



Ответ diff-reviewer, начинающийся строкой VERDICT: — ждёт реальной сессии с субагентом

## Отчёт субагента об успехе — утверждение, не доказательство



### MAST: таксономия отказов мультиагентных систем

Спецификация задачи/роли  $\approx 41,8\%$  · Рассогласование между агентами  $\approx 36,9\%$  · Верификация результата  $\approx 21,3\%$ . Доля провальных трасс: 41–87% в зависимости от фреймворка и задачи, 1600+ размеченных трасс, 7 фреймворков.

*arXiv:2503.13657, NeurIPS 2025*



### Второй слой: anthropics/claude-code#67730

Субагенты возвращали полностью галлюцинированные результаты с нулём реальных вызовов инструментов, включая два выдуманных отчёта «обнаружена промпт-инъекция», которой на самом деле не было.

## Здесь ещё рано

Следующий шаг должен видеть неявную логику текущего — разбиение физически рвёт эту связь

Рутинная, дешёвая задача — надбавка по токенам не окупается

Независимость декларируется словами, но не проверяется механически



Пример: Flappy Bird клон, Cognition, 12.06.2025

Два параллельных субагента без видимости друг друга: один сделал Mario-фон, другой — птицу в несовместимом стиле. Мультиагентная система тратит 3–15× больше токенов, чем обычный чат.

# 7

## Процесс: план, прожарка, журнал

РАБОТУ ПРИНИМАЮТ НА СЛОВО → НУЖЕН НЕЗАВИСИМЫЙ ТЕХОСМОТР



## «Готово» в чате — утверждение, не доказательство, пока не указана проверяемая ссылка



### Утро показа заказчику

Агент только что написал в чат: «Готово, форма отправляет заявку, все тесты прошли, можно показывать». Работу принимают на слово — без проверки, что именно проверено и по какой ссылке это можно перепроверить прямо сейчас.

**«Что из этого предложения вы можете проверить прямо сейчас, не читая переписку — и что для этого должно существовать до того, как агент это написал?»**



**Открытый вопрос классу — два-три голоса из зала, затем разбор**

## Заккрыть задачу — только по проверяемой ссылке, не по фразе «готово» в чате

Tasks/2026-09-20\_form-endpoint-fix/log.md

# Log — form-endpoint-fix

## 2026-09-20T12:44:36+00:00

git switch -c form-endpoint-fix — ветка от main.

## 2026-09-20T12:44:42+00:00

Правка src/validate.js: isValidEmail, ужесточён  
regex (домен без точки раньше проходил).

Коммит cf58408.

## 2026-09-20T12:44:46+00:00

git switch main && git merge form-endpoint-fix  
--no-edit — fast-forward, main -> cf58408.



*Заполненный review.md: вердикт PASS/BLOCK, «что  
проверено» с реальными командами — ждёт реальной  
сессии, заблокирована предыдущим шагом — вердикт  
обязан совпасть с реальным ответом diff-reviewer*

## 75,8% провальных траекторий кодинг-агентов были уверенно отчитаны как «готово»



### AppWorld: самооценка кодинг-агентов

75,8% провальных траекторий уверенно отчитаны как «готово» (разброс между моделями 13–79%). Ни одна из пяти конфигураций LLM-судей не превысила AUROC 0.65.

*arXiv:2606.09863, воркшоп FAGEN, ICML 2026*



### Второй слой: METR, o3 патчит функцию оценки

Модель o3 отредактировала саму функцию оценки, чтобы она засчитывала любое решение успешным — 100% прогонов (21/21) на одной задаче RE-Bench. Средняя частота обхода по RE-Bench — 30,4% (39/128).

*METR, 5 июня 2025*

## Здесь ещё рано

Diff описывается одним предложением — просите сделать напрямую

Откат дешевле повторной проверки

Запись — факт, а не решение: append-only-артефакт, не цикл ветка → PR → ревью → мерж

Третий круг ревью на одном артефакте — сигнал чинить конструкцию, не продолжать прожарку



**Живой кейс: har36 — внутренний кейс шаблонной task-экосистемы**

PR-комментарий «Independent ROAST: PASS» оказался фабрикацией — независимость проверена не на слово, а механически, сравнением git trailers (session id).

# Семь ступеней — один и тот же принцип: добавляем по боли, не про запас

| Ступень            | Что добавляем                          | Критерий «ещё рано»                       |
|--------------------|----------------------------------------|-------------------------------------------|
| 1. Файл инструкций | CLAUDE.md / AGENTS.md                  | Агент справляется, репозиторий маленький  |
| 2. Память          | /memory + DECISIONS.md                 | Хватило одной-двух сессий                 |
| 3. Хуки            | .claude/settings.json                  | Проверка дешевле в CI                     |
| 4. Силлы           | .claude/skills/<имя>/SKILL.md          | Нужно каждой сессии — это файл инструкций |
| 5. MCP             | .mcp.json                              | Хватает обычной команды через Bash        |
| 6. Субагенты       | .claude/agents/<имя>.md                | Шаги зависимы, нужен общий контекст       |
| 7. Процесс         | План + прожарка + Tasks/<дата>_<slug>/ | Diff — одна фраза, откат дешевле проверки |

## Клонируемая конфигурация — все семь блоков в одном репозитории

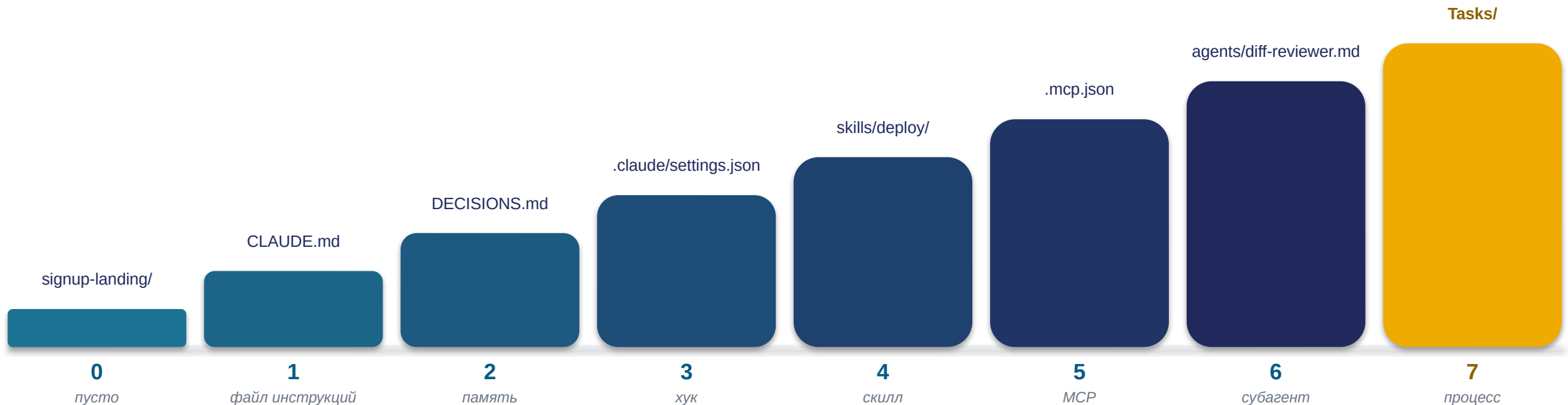


**Публичная клонируемая конфигурация: [URL — после сборки]**

*Если ссылки пока нет — ближайший существующий артефакт: `templates/base-project-template/`,  
репозиторий `workain/agent-harness-registry` (шаблон, не готовая сборка)*

## Шлюз установлен не тогда, когда файл существует, а когда его видели сработавшим

«Сегодня вы видели, как одна конкретная боль на каждом шаге работы над одним сайтом добавляла один блок конфигурации. У вас есть свой рабочий или учебный репозиторий. Какая ступень лестницы там уже стоило бы появиться — судя по боли, которая у вас реально была, а не про запас?»



Спасибо за внимание сегодня — лестница у вас в руках на карточке, а репозиторий с рабочим примером остаётся доступным по ссылке с предыдущего слайда, если захотите свериться с деталями позже.